Smart Resume ↔ Job Description Matcher

TECHNICAL REPORT

Sébastien LEVESQUE

Link to the Github repository:
https://github.com/TheSese1/Smart_Resume_to_Job_Matcher
Live presentation of the app: youtube.com/watch?v=WF9D-Tiwn_Y
Live presentation of the powerpoint: https://youtu.be/y_6_A5GtRrA

Introduction and Problematic Description

Recruitment pipelines increasingly rely on semi-automated screening procedures to reduce workloads and accelerate candidate filtering. However, traditional matching systems primarily perform lexical keyword extraction or rule-based scoring, which fail to capture semantic equivalence between job requirements and candidate resume. This leads to mismatches for candidate resumes not using the same vocabulary as the job description, despite semantic similarity between the two.

Our objective was to design, implement, and evaluate a system capable of matching resumes to job descriptions using language-model–based semantic representations and LLM-driven explanations. The system must :

1. Support multiple file formats (PDF, DOCX, TXT)
2. Extract the main information from the documents
3. Perform semantic matching and ranking beyond keyword overlap
4. Generate interpretable explanations for the match quality
5. Maintain acceptable latency for interactive usage

Ultimately, this system aims to explore whether LLM-based information retrieval can improve resume reviewing efficiency while reducing biases created by manual reviewing.

Architecture Overview

We implemented a modular architecture composed of four main subsystems:

1. **Document Ingestion**
   Converts heterogeneous document formats (PDF, DOCX, TXT) into a string format.

2. **Extraction and Normalization**
   Extracts data and structures it into a standardized output format.
3. **Embedding-Based Semantic Matching**
   Computes embeddings for both resumes and job descriptions, enabling similarity search.
4. **LLM Explanation Layer**
   Generates natural-language reasoning to justify matches produced by semantic matching.
5. **User Interaction Layer (Streamlit App)**
   Supports upload, visualization, ranking results, and latency evaluation.

1. Components

- **Ingestion** supports PDF, text or docx files as well as the initial dataset format, tables with descriptions.
- **Normalization agent** supports both job descriptions and resumes and ensures standardized output.
- **Embedding Engine** supports two embedding backends:
  - BGE embedding model
  - Nomic embedding model
- **Matching Engine** supports cosine similarity or Euclidean distance scoring.
- **Explanation Agent** invokes an LLM to justify why two documents align.
- **Smart Resume Notebook** provides an example of usage of such functions outside the main application.
- **Evaluation Notebook** provides empirical analysis and metrics.

2. Deployment Considerations

The architecture supports incremental loading and caching to reduce cold-start overhead, since embedding loading and LLM initialization dominate initial inference time. The system can operate both in local batch mode (notebook) and interactive mode (web interface).

Pipeline description

The processing pipeline is designed to support two symmetric directions:

- Upload resume → match against job descriptions

- Upload job description → match against resumes

Both directions share the same underlying pipeline:

Step 1: Document Ingestion

Input documents may include PDF, DOCX and TXT files.

This step extracts raw textual content using format-specific parsing and cleans it by removing useless spaces.

Step 2: Text Normalization

To reduce noise and increase alignment, we normalize documents into structured JSON formats using an LLM. For example:

Resume → skills (array), experience (array), ...

Job description → job title (string), required skills (array), ...

Normalization mitigates variation such as bullet formatting, casing, OCR artifacts, and inconsistent section headings.

However, as LLM are inherently probabilistic, a second formalization phase is necessary, this time using python code, for a standardized output.

Step 3: Embedding Generation

Based on user selection, either BGE or Nomic embeddings transform normalized embedding text into vectors. This enables semantic similarity search.

Step 4: Matching & Ranking

Similarity scoring is computed between resume vectors and job vectors, depending on the input: 1 resume with all job vectors or the opposite. Ranking sorts candidate matches by decreasing similarity and returns top-K.

Step 5: LLM-Based Explanation

For each returned match, the LLM receives both normalized job description and candidate profile as well as their similarity score and produces a justification of the match.

This ensures interpretability beyond a numeric score.

Step 6: Display & Export

A Streamlit front-end presents ranked matches with expandable explanations.

The symmetric nature of the pipeline supports both hiring and job-seeking use cases.

Evaluation methodology

Evaluation aimed to assess both performance and semantic quality. Since there were no pre-existing labeled matches between resumes and job descriptions, we could not compute traditional accuracy metrics.

Instead, we will personally evaluate the agents' outputs based off of three evaluation dimensions: **Latency Analysis**, **Ranking Alignment and Explanation Coherence.**

1. Latency Analysis

Latency is measured by the application and given as a report at the end. Different experiments were done using this feature such as:

- Per-step bottlenecks
- Cold vs Hot start effects
- Sensitivity to Top-K
- Document size effects

2. Ranking Alignment

Since no labeled dataset exists linking specific resumes to jobs, we performed manual evaluation:

- For one resume, we manually ranked five relevant job descriptions.
- For one job description, we manually ranked five resumes.
- The model generated ranked lists for both.
- We compared list agreement and observed qualitative differences.

We also studied the distribution of similarities, for one example resume with all job descriptions directly through the application.

3. Explanation Coherence

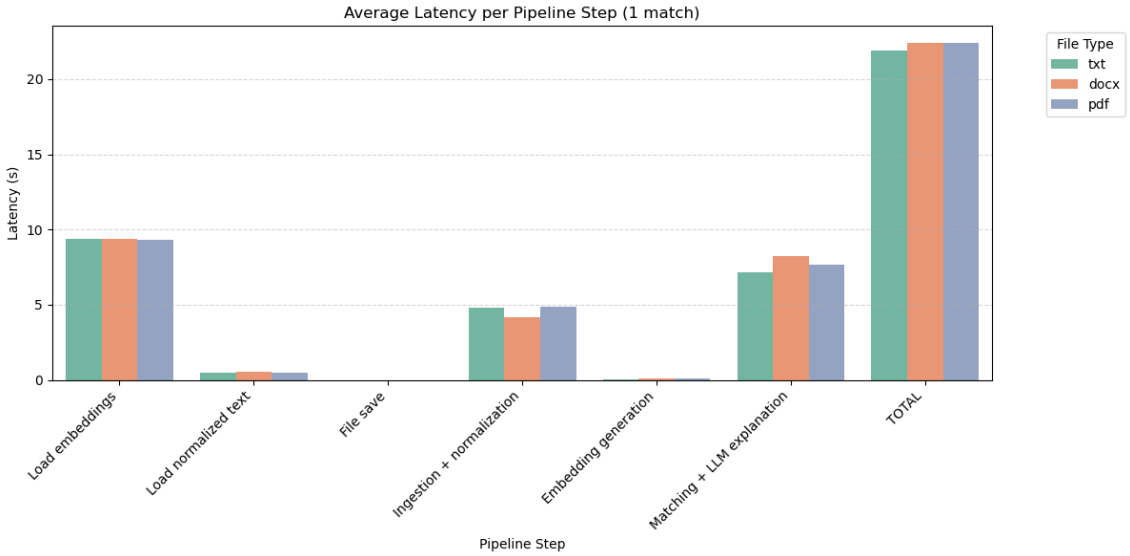
Explanations were assessed subjectively based on:

- Relevance compared to personal results
- Factual correctness (staying grounded in the document)
- Clarity (structured justification)

Results

1. Latency per Pipeline Step

The following plot summarizes the average latency for each pipeline step across three document formats.

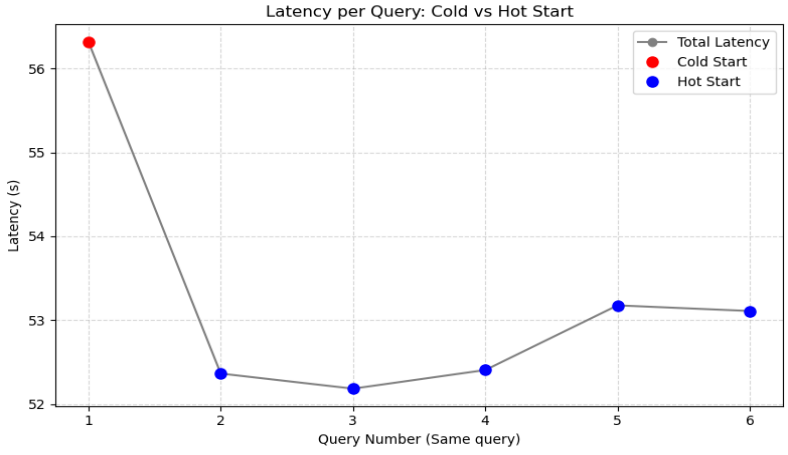


Observations :

- Embedding loading and LLM explanation dominate total latency.
- Ingestion time is the step that varies the most with file format due to parsing overhead.
- Total runtime remains roughly uniform (~22–23 s), demonstrating that file format is not the primary bottleneck.

2. Cold vs Hot Start Effects

Repeated queries on the same matching task were executed sequentially. First query triggers model and embedding initialization.

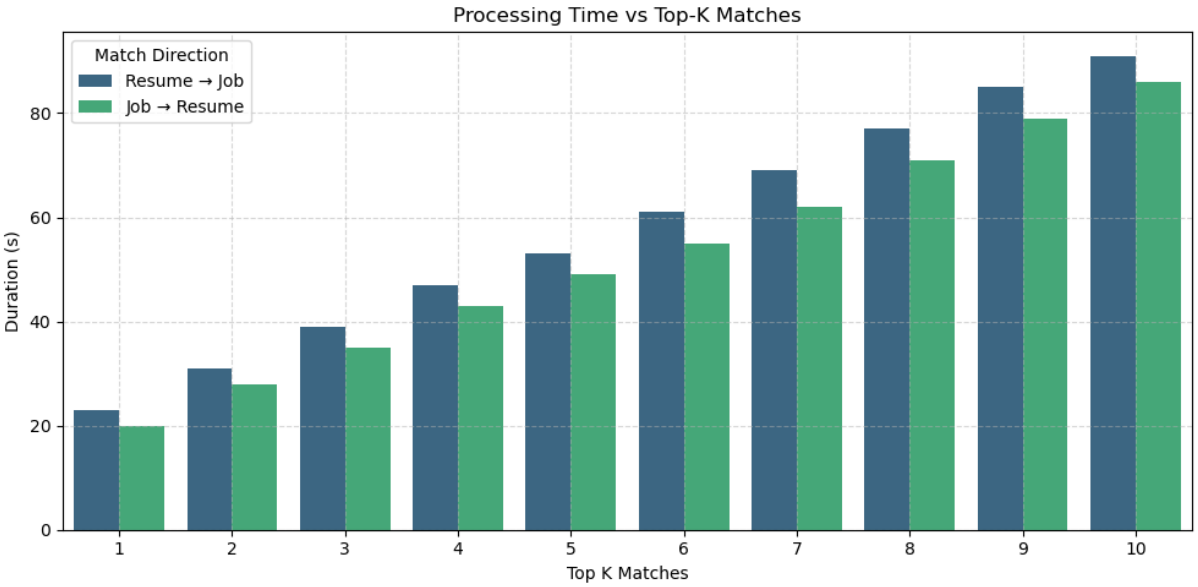


Observations :

- Cold start latency ~56 s
- Hot start latency stabilizes at ~52–53 s
- Relative reduction ~4 s after warm-up
- Hot start variance is minimal across runs
- This suggests caching and initialization are key contributors to cold start overhead but that they are largely negligeable for batch queries.

3. Sensitivity to Top-K

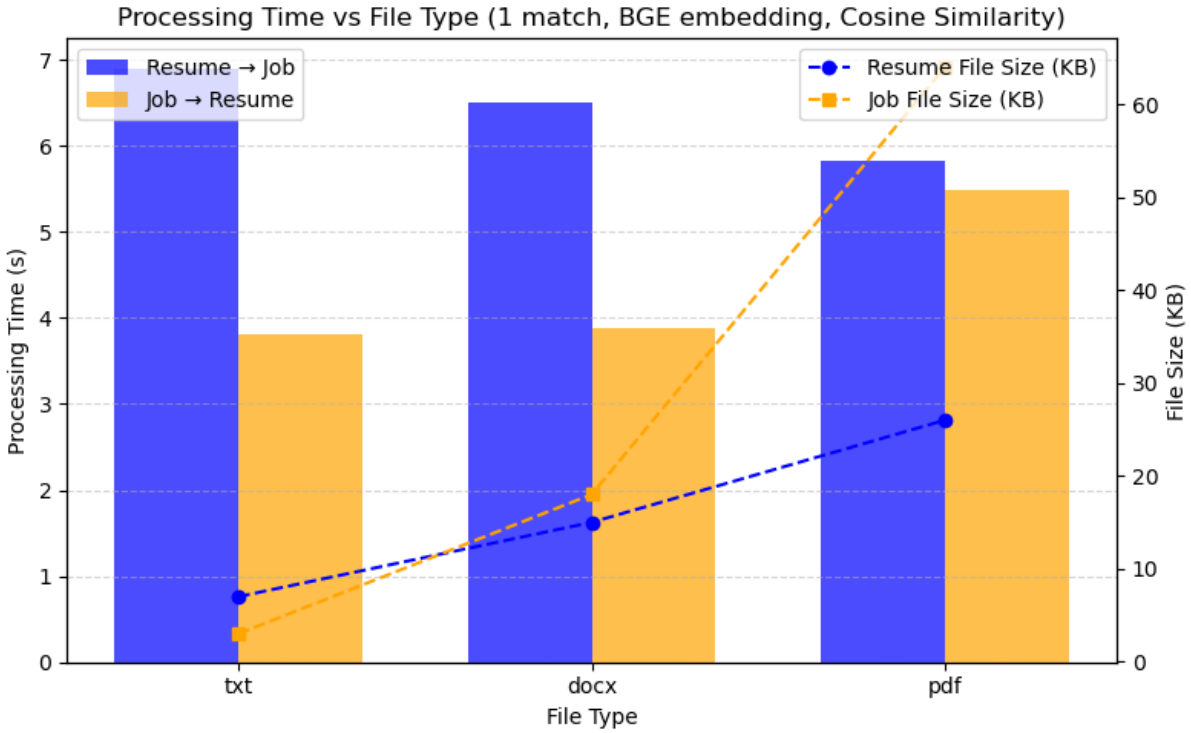
Repeated queries on the same matching task were executed sequentially with different values for the number of matches.



The plot seems to be linear. That is not surprising as resumes and jobs seem to all have approximately the same size. However, resumes are also proportionally longer than job descriptions, giving this relatively stable step between the bars.

4. Document size effect

Queries were done on documents of different sizes and types (txt, docx and pdf) to study their effect on processing time. Here is the result:



Even if it was done on very little example, we can see that the document type is highly correlated with the document size, but apparently not with the processing time. Indeed, the job file size seems indeed positively correlated to processing time. However, for the resume files, weirdly, they seem to be negatively correlated.

The main criticism we can make about this study is that only one file of each type was considered. Maybe taking a means across many documents could be more beneficial.

5. Ranking Evaluation

Example manual vs model ranking:

Human: (6, 4, 79, 10, 61)
Model: (61, 79, 6, 10, 4)

Analysis showed Spearman correlation of -0.5, meaning that there is partial overlap, but that said partial overlap is very insignificant.

With no labeled dataset, ranking was ensured qualitatively for top-matches, and for only one sample. This makes the study very fragile and not reliable. For a more significant study, a ranking done by multiple professionally trained talent acquisitors would have been better (but I didn't have access to that).

6. Explanation Quality

On a few given examples, the explanation given seemed related to the input given, but the LLM response would often hallucinate on given information about the matched elements.

This quality assessment, being purely qualitative and done alone, bias is obvious. Thus, this evaluation should be taken as a pure example methodology and not an assessment of the model itself.

Challenges

Several challenges emerged during implementation:

1. **Absence of Ground Truth Labels**
   Without pre-labeled resume–job matches, evaluation required manual or qualitative approaches.
2. **Unrealistic dataset**
   I decided to go with volume of data instead of quality. Given that the initial input data of resumes and job descriptions was generated, the texts were not very satisfying.
3. **LLM output normalization**
   Ensuring consistently normalized output from the LLM via prompt engineering alone was impossible given the probabilistic nature of LLMs, requiring multiple attempts to get the required architecture.
4. **Data volume**
   The volume of processed data significantly impacted the speed at which the project would progress.
5. **Latency Constraints**
   LLM explanation introduces overhead unsuitable for low-latency environments.
6. **Semantic Ambiguity**
   Some matches require domain inference beyond text (e.g., implicit certifications, seniority).
7. **Noisy Skill Extraction**
   Extracting relevant skills without over-tagging is challenging for generic resumes.

Future work

Potential extensions include:

1. **Supervised Fine-Tuning**
   Creating a labeled dataset to benchmark and optimize ranking alignment based on real data from multiple sources for diversity (online resumes, LinkedIn, ...)
2. **Hybrid Retrieval Models**
   Combining embedding-based retrieval with learned re-ranking layers for improved precision.
3. **Advanced Agent Collaboration**
   Expand multi-agent LLM collaboration for deeper skill and personality assessments.
4. **Skill Ontology Integration**
   Mapping extracted skills to standardized taxonomies (e.g., ESCO, O*NET).
5. **Model Distillation for Latency Reduction**
   Reduce LLM footprint for interactive systems or edge deployment.
6. **Bias and Fairness Evaluation**
   Assess potential demographic or linguistic biases in ranking outcomes.
7. **Dynamic Market Insights**
   Integrate real-time labor market data for dynamic job requirement updates and trend analysis.
8. **Explainability Dashboard**
   Develop an intuitive explainability dashboard for candidate and job insights.

Conclusion

We designed and implemented a semantic matching system for resumes and job descriptions leveraging embedding models and LLM-based explanations. The system demonstrates the feasibility of using modern language-model representations to move beyond simple keyword-based matching, enabling more flexible and context-aware candidate–job alignment. While latency remains manageable for interactive use and the modular architecture supports both batch and real-time workflows, evaluation revealed significant limitations, including lack of ground-truth labels, probabilistic variability in LLM outputs, and challenges in ranking alignment and explanation reliability.

Despite these limitations, the project highlights the potential of combining embeddings and LLM explanations to enhance interpretability and support recruitment decisions.

Future work should focus on creating labeled datasets, incorporating domain-specific skill ontologies, optimizing model efficiency for low-latency deployment, and systematically evaluating fairness and bias. Overall, this study provides a foundation for intelligent, semantically aware resume–job matching systems and underscores the importance of structured evaluation frameworks to reliably measure performance and interpretability in applied recruitment AI.